B.TECH CSE — 5th SEMESTER LECTURE NOTES

Chapter 3
Search and Problem Solving

Based on: *Artificial Intelligence and Machine Learning*

Pradeep Singh, Tapan K. Gandhi, Balasubramanian Raman



Contents

3 Search and Problem Solving **1**

3.1 Introduction . . . 1
3.2 Problem Space and Search: Formulating Problems . . . 2
3.2.1 Understanding Problem Space . . . 2
3.2.2 Formulating Problems as Search Tasks . . . 3
3.2.3 State-Space Graph, Search Tree and Frontier . . . 4
3.2.4 Evaluating Problem Characteristics . . . 5
3.3 Search Algorithms . . . 6
3.3.1 Uninformed Search . . . 6
3.3.2 Informed Search . . . 12
3.3.3 Comparison of Search Strategies . . . 16
3.4 Optimization . . . 16
3.4.1 Local Search Techniques . . . 16
3.5 Adversarial Search . . . 19
3.5.1 Game Theory in AI . . . 20
3.5.2 Minimax Search Algorithm . . . 20
3.5.3 Alpha–Beta Pruning . . . 22
3.6 Example Problems in Search and Optimization . . . 24
3.6.1 One Graph, Four Search Strategies . . . 25
3.6.2 Job Scheduling with Constraints . . . 26
3.6.3 Travelling Salesperson Problem (TSP) . . . 26
3.6.4 Small Practice Problems with Answers . . . 27
3.7 Revision and Self-Study . . . 28
3.7.1 Common Mistakes . . . 28
3.7.2 Short Revision Questions . . . 29
3.8 Quick Revision Sheet . . . 30

Search and Problem Solving

What this chapter covers: Turning a task into a search problem; exploring it with BFS, DFS, UCS, Greedy Best-First Search and A*; improving candidate solutions with local search; satisfying constraints; and choosing moves with minimax and alpha–beta pruning.

Why it matters: A robot choosing a route, a puzzle solver choosing a move, and a game-playing program choosing a strategy all need to compare alternatives. Search gives a systematic way to make these choices.

3.1 Introduction

Suppose a robot must reach a classroom but several corridors are available. It knows the starting room, the connected rooms and the destination. It must decide *which sequence of moves* to follow. This is a search problem.

Definition 1: Search: Search is the process of finding a sequence of valid actions that takes an agent from an **initial state** to a state satisfying a **goal test**. A solution is such a sequence; an optimal solution has the smallest total cost under the chosen cost measure.

Two questions should be kept separate:

- **Can we reach the goal?** This asks for any valid solution.
- **What is the cheapest way to reach it?** This asks for an optimal solution.

The fewest moves and the least cost coincide only when all moves have the same positive cost.

Search family	Main idea
Uninformed	Use the problem definition without an estimate of the distance to the goal: BFS, DFS, UCS.
Informed	Use a heuristic to guide exploration: Greedy Best-First Search and A*.
Local	Improve a candidate state when the final state matters more than the path: hill climbing, simulated annealing, genetic algorithms.
Adversarial	Account for an opponent's choices: minimax and alpha–beta pruning.

3.2 Problem Space and Search: Formulating Problems

3.2.1 Understanding Problem Space

The **state space** is the collection of possible configurations. A state should contain the information needed to decide which actions are legal and what those actions will do. It may be a city name, a pair of coordinates, a board configuration or a vector of values.

Solved Problem 1: State representation and formulation of the 8-puzzle The puzzle has eight numbered tiles and a blank in a 3×3 board. For example,

$$s = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 5 & 8 \end{bmatrix}, \qquad s_g = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix},$$

where 0 denotes the blank.

- **State:** The complete board arrangement.
- **Actions:** Move the blank up, down, left or right, when legal.
- **Transition:** Swap the blank with an adjacent tile.
- **Goal test:** Does the board match s_g?
- **Cost:** One unit per move.

A solution is: move the blank down, then right. Two moves give the goal board. Moving down swaps the blank with tile 5; moving right swaps it with tile 8.

Actions and transitions. Let $A(s)$ be the legal actions in state s. The deterministic transition model $T(s, a)$ tells us the next state after action a. The notation $T : S \times A \to S$ is shorthand; only legal state–action pairs are used.

Solved Problem 2: Robot navigation on a grid Represent a position by $s = (x, y)$. The legal moves have transitions

$$T((x, y), a) = \begin{cases} (x-1, y), & a = \text{left}, \\ (x+1, y), & a = \text{right}, \\ (x, y+1), & a = \text{up}, \\ (x, y-1), & a = \text{down}. \end{cases}$$

If a move leaves the grid or enters an obstacle, it is excluded from $A(s)$. From $(2, 1)$, a legal right move reaches $(3, 1)$.

3.2.2 Formulating Problems as Search Tasks

Definition 2: Search problem A useful formal description is the six-tuple

$$\mathcal{P} = (S, A, T, s_0, S_g, C).$$

Here S is the state space, A the actions, T the transition model, s_0 the initial state, S_g the goal states, and $C(s, a)$ the cost of a legal action.

For a path $\pi = (s_0, a_1, s_1, \ldots, a_k, s_k)$,

$$g(\pi) = \sum_{i=0}^{k-1} C(s_i, a_{i+1}), \qquad s_k \in S_g.$$

The goal test checks whether the current state belongs to S_g.

Five components or six? Some descriptions list initial state, actions, transition model, goal test and path cost as *five* components because the state space is implicit. The six-tuple makes the state space explicit. Both describe the same search problem.

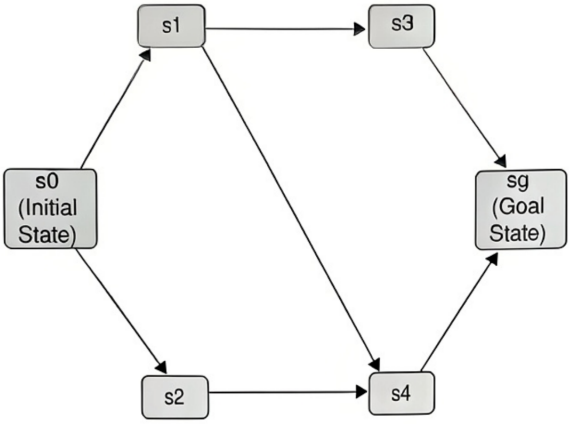


Figure 3.1: A supplied state-space graph: arrows are valid transitions from the initial state to a goal.

In Figure 3.1, both $s_0 \to s_1 \to s_3 \to s_g$ and $s_0 \to s_2 \to s_4 \to s_g$ reach the goal. Without edge costs, we can compare their numbers of moves but cannot decide which has the lower travel cost.

Solved Problem 3: Formulate the water-jug problem There is a 4-litre jug and a 3-litre jug. Measure exactly 2 litres.

- **State:** (x, y), where $0 \le x \le 4$ and $0 \le y \le 3$ are the amounts in the two jugs.
- **Initial state:** $(0, 0)$.
- **Actions:** Fill a jug, empty a jug, or pour until the source is empty or the destination is full.
- **Goal test:** $x = 2$ or $y = 2$.
- **Cost:** One unit per action.

One valid sequence is

$$(0, 0) \to (0, 3) \to (3, 0) \to (3, 3) \to (4, 2).$$

The actions are: fill the 3-litre jug, pour into the 4-litre jug, refill the 3-litre jug, and pour one litre into the 4-litre jug. Two litres remain in the smaller jug.

3.2.3 State-Space Graph, Search Tree and Frontier

The **state-space graph** describes the problem. The **search tree** records paths considered by an algorithm. A state can appear in several search nodes when reached by different paths.

Term	Meaning
Search node	State plus parent, incoming action, depth and path cost.
Frontier	Generated nodes waiting to be selected; also called the open list.
Expanding a node	Applying its legal actions to generate successors.
Explored set	States already expanded; also called the closed list.
Discovered set	States already seen, including states waiting in the frontier.

Generating is not expanding. When S produces child A, A has been *generated*. Only when A is selected and its successors are produced is it *expanded*. A selected goal can be returned without expanding it. State the counting convention in node-count questions.

3.2.4 Evaluating Problem Characteristics

Characteristic	Meaning	Example
Deterministic	A legal action has a predictable successor.	8-puzzle.
Stochastic	An action may have several outcomes with probabilities.	A robot whose wheel may slip.
Single-agent	One agent makes the decisions.	Maze solving.
Multi-agent	Other agents influence the outcome.	Two-player chess.
Static	The environment does not change while the agent plans.	A printed maze.
Dynamic	The environment changes while the agent plans.	Driving through traffic.

Evaluation criteria. A search algorithm is ***complete*** if it is guaranteed to find a solution when one exists under stated assumptions. It is ***optimal*** if the returned solution has minimum cost. Time measures the search work; space measures the memory used.

Throughout the notes, b denotes the **branching factor**, d the **depth** of the shallowest goal, and m the **maximum search-tree depth**. A finite graph can generate an infinite tree if cycles are repeatedly followed.

3.3 Search Algorithms

3.3.1 Uninformed Search

Definition 3: Uninformed search An uninformed or blind search strategy uses the initial state, legal actions, goal test and action costs, but has no additional estimate of the remaining cost to the goal.

Each strategy repeatedly selects a node from the *frontier*, checks or detects the goal, and generates successors. The main difference is **which waiting node is selected next**. Unless stated otherwise, successors are considered left to right and ties retain insertion order.

1. Breadth-First Search (BFS)

BFS explores by depth: the start, then states one move away, then states two moves away, and so on. It uses a **FIFO queue**: first in, first out.

Algorithm 3.1 Breadth-First Search (BFS)

Input: Initial state s_0, Goal test S_g, Transition function T
Output: Solution path π or failure
Initialize an empty queue Q
Insert $(s_0, \text{empty path})$ into Q
Initialize an empty set Visited
while Q is not empty **do**
  $(s, \pi) \leftarrow$ dequeue from Q
  if $s \in S_g$ **then**
    return π {Goal found, return solution}
  end if
  for each action $a \in A$ applicable to s **do**
    $s' \leftarrow T(s, a)$
    if $s' \notin$ Visited **then**
      Insert $(s', \pi + a)$ into Q
      Add s' to Visited
    end if
  end for
end while
return failure {No solution found}

Figure 3.2: Algorithm for Breadth-First Search

Why mark a state when adding it? Otherwise, two parents may enqueue the same state before either copy is processed. Marking the start also prevents a cycle from reintroducing it. Parent links reconstruct the path by working backwards from the goal.

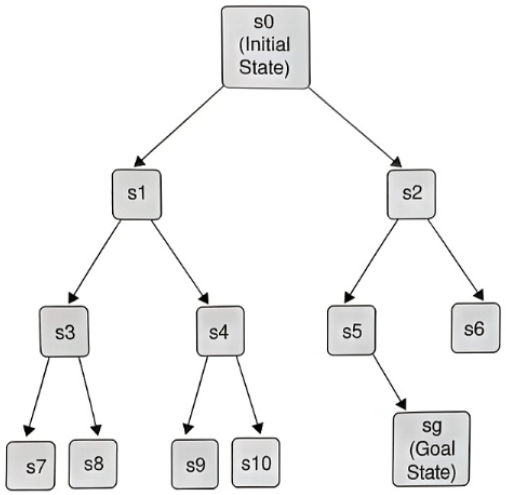


Figure 3.3: The supplied BFS tree. Read children left to right and visit levels in increasing depth.

If the complete tree in Figure 3.3 is traversed, its order is

$$s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_g.$$

Algorithm 3.1 stops earlier: generating s_g while expanding s_5 finds the solution $s_0 \to s_2 \to s_5 \to s_g$.

Solved Problem 4: BFS pathfinding with the queue shown Start at $n = 1$, reach $n = 10$, and allow $n \to n + 3$ followed by $n \to n - 2$ as the action order. Each action costs one.

Stage	New states	Queue, front on the left
Initially	Start at 1	$[1]$
Expand depth 0	From 1: $4, -1$	$[4, -1]$
Expand depth 1	From 4: $7, 2$; from -1: -3	$[7, 2, -3]$
Begin depth 2	From 7: generate 10, a goal	Stop and return the path.

The state 2 is generated from both 4 and -1, but is queued once. The returned path is

$$\boxed{1 \to 4 \to 7 \to 10}, \qquad \text{cost} = 3.$$

Two actions cannot suffice: the largest possible increase in two actions is 6, whereas the required increase is 9.

Properties. BFS is complete when branching is finite and a goal is at finite depth. It returns a shallowest goal and is cost-optimal when every action has the same positive cost. Its main limitation is the large frontier.

For the version that tests the goal on generation, worst-case time and space are $O(b^d)$. The number of nodes through depth d in a full tree is

$$N = 1 + b + \cdots + b^d = \frac{b^{d+1} - 1}{b - 1} \quad (b > 1).$$

For $b = 1$, $N = d + 1$. A version that waits until dequeueing to test the goal can generate depth-$d + 1$ nodes before it stops, giving the more conservative $O(b^{d+1})$ bound.

2. Depth-First Search (DFS)

DFS follows one branch as deeply as possible, then backtracks to a waiting alternative. It uses a **LIFO stack**: last in, first out. Recursion uses the call stack for the same purpose.

Algorithm 3.2 Depth-First Search (DFS)

```
1: Input: Initial state s_0, Goal test S_g, Transition function T
2: Output: Solution path π or failure
3: Initialize an empty stack S
4: Push (s_0, empty path) onto S
5: Initialize an empty set Visited
6: while S is not empty do
7:     (s, π) ← pop from S
8:     if s ∈ S_g then
9:         return π
10:    end if
11:    for each action a ∈ A applicable to s do
12:        s' ← T(s, a)
13:        if s' ∉ Visited then
14:            Push (s', π + a) onto S
15:            Add s' to Visited
16:        end if
17:    end for
18: end while
19: return failure
```

Figure 3.4: Algorithm for Depth-First Search

Push right first to visit left first. If the children are A then B, pushing B and then A leaves A on top. State whether the top of a drawn stack is on the left or the right.

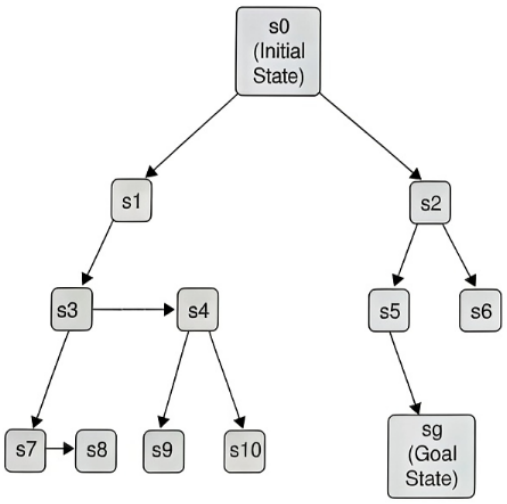


Figure 3.5: The supplied DFS graph. Its horizontal arrows are transitions, so follow the actual arrows when tracing the search.

Taking the visually left branch first in Figure 3.5, the selection order up to the goal is

$$s_0, s_1, s_3, s_7, s_8, s_4, s_9, s_{10}, s_2, s_5, s_g.$$

The goal's parent chain is $s_0 \to s_2 \to s_5 \to s_g$. Nodes selected earlier on other branches are *not* part of this solution path.

Properties. DFS is not generally optimal. Without duplicate detection, cycles can cause looping; even with it, an infinite branch can delay a goal forever. On a finite reachable graph, the graph version above is complete.

For a finite search tree with maximum depth m, time is $O(b^m)$ and stack space is $O(bm)$: the current path plus waiting siblings. A graph version also stores discovered states, which can require $O(|V|)$ memory. Thus "DFS stores only the path" is an incomplete description.

Solved Problem 5: Counting nodes fairly: BFS versus DFS Consider a full tree with $b = 3$ and maximum depth $m = 4$. Its node count is

$$1 + 3 + 9 + 27 + 81 = 121.$$

With no goal test, both traversals visit all 121 nodes; only their order differs. BFS processes all 40 nodes at depths 0–3 before selecting a depth-4 node. This 40 is *not* its worst-case total visit count.

If the only goal is the last leaf in the stated order, a dequeue-testing BFS and DFS may each select all 121 nodes, with the goal not expanded. Generation-testing BFS processes all 40 internal nodes in the worst case and can generate all 121 nodes before returning the final leaf. Always distinguish *generated*, *selected* and *expanded* counts.

3. Uniform-Cost Search (UCS)

When steps have different costs, BFS's depth ordering is insufficient. UCS selects the node with the **smallest accumulated path cost**

$$g(n) = \text{cost of the path from the start to } n.$$

Its frontier is a **priority queue ordered by** g. A cheaper route to an already discovered state must improve its stored cost and parent.

Algorithm 3.3 Uniform-Cost Search (UCS)

```
Input: Initial state s_0, Goal test S_g, Transition function T, Cost function C
Output: Solution path π or failure
Initialize an empty priority queue PQ
Insert (s_0, empty path, 0) into PQ with priority 0
Initialize an empty set Visited
while PQ is not empty do
    (s, π, g) ← dequeue from PQ
    if s ∈ S_g then
        return π
    end if
    for each action a ∈ A applicable to s do
        s' ← T(s, a)
        g' ← g + C(s, a)
        if s' ∉ Visited or g' < g(s') then
            Insert (s', π + a, g') into PQ with priority g'
            Add s' to Visited
        end if
    end for
end while
return failure
```

Figure 3.6: Algorithm for Uniform Cost Search

This pseudocode permits an improved entry to be added and skips its older copy later. An implementation with *decrease-key* can instead update the existing queue entry. Both represent the same best-cost search.

Solved Problem 6: Trace UCS and improve a previously discovered route Edges are $S \to A$ (1), $S \to B$ (4), $A \to B$ (2), $A \to C$ (5), and $B \to C$ (1). The goal is C.

Step	Selected	g	Frontier after processing; best entries only
1	S	0	$[A:1,\ B:4]$
2	A	1	$[B:3,\ C:6]$; improve B from 4 to 3.
3	B	3	$[C:4]$; improve C from 6 to 4.
4	C	4	Goal selected; stop.

The solution is $\boxed{S \to A \to B \to C}$ with cost $1+2+1=4$. Old entries with costs 4 for B and 6 for C are ignored when removed.

For UCS, test the goal when it is removed from the priority queue. A goal generated on an expensive route may later be reached more cheaply. Returning it immediately would lose the optimality guarantee.

Guarantees. On a finite graph with duplicate handling, UCS is complete and optimal with nonnegative costs. For an infinite search space, the usual completeness guarantee also assumes finite branching and a positive lower bound $C(s, a) \geq \epsilon > 0$. Infinitely many zero-cost alternatives can prevent a positive-cost goal from being selected.

Its search-tree time and space are commonly bounded by

$$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right),$$

where C^* is the optimal solution cost and ϵ is the cost for each step. The exponent measures how many cheap steps may be explored before that cost is exceeded.

3.3.2 Informed Search

Definition 4: Heuristic function $h(n)$ estimates the remaining cost of the cheapest path from node n to a goal. In contrast, $g(n)$ is the actual cost of the particular path already found from the start to n.

1. Greedy Best-First Search (GBFS)

GBFS selects the frontier node with the smallest **estimated remaining cost**:

$$\boxed{f(n) = h(n)}.$$

It ignores the cost already paid to reach the node. A node that looks close to the goal may lie on an expensive route.

Algorithm 3.4 Greedy Best-First Search (GBFS)

Require: A search problem $\mathcal{P} = (S, A, T, s_0, S_g)$, heuristic function $h(s)$
Ensure: Solution path π from s_0 to a goal state s_g

```
Initialize an empty priority queue Q
Insert s_0 into Q with priority h(s_0)
Initialize an empty set Explored
while Q is not empty do
    Remove the node s with the lowest h(s) from Q
    if s ∈ S_g then
        return Solution path by tracing back predecessors
    end if
    Add s to Explored
    for each action a ∈ A(s) do
        Compute successor s' = T(s, a)
        if s' ∉ Explored then
            Insert s' into Q with priority h(s')
        end if
    end for
end while
return No solution found
```

Figure 3.7: Algorithm for Greedy Best-First Search

Guarantees. GBFS is not generally optimal. On a finite graph with duplicate detection, it is complete because the waiting alternatives are eventually considered. In an infinite space, a never-ending sequence of attractive heuristic values can prevent it from reaching a goal elsewhere. This is different from hill climbing, which discards the alternative frontier branches.

2. A Search Algorithm*

A* balances the cost already paid with the estimated cost still required:

$$\boxed{f(n) = g(n) + h(n)}.$$

For example, if the frontier contains $A : (g, h) = (4, 6)$, $B : (2, 11)$ and $C : (7, 2)$, their f-values are 10, 13 and 9. A* selects C.

Algorithm 3.5 A* Search Algorithm

Require: A search problem $\mathcal{P} = (S, A, T, s_0, S_g)$, cost function $g(s)$, heuristic function $h(s)$
Ensure: Solution path π from s_0 to a goal state s_g

```
Initialize a priority queue Q and insert s_0 with priority f(s_0) = g(s_0) + h(s_0)
Initialize an empty set Explored
Set g(s_0) = 0
while Q is not empty do
    Remove the node s with the lowest f(s) from Q
    if s ∈ S_g then
        return Solution path by tracing back predecessors
    end if
    Add s to Explored
    for each action a ∈ A(s) do
        Compute successor s' = T(s, a)
        Compute g(s') = g(s) + C(s, s')
        Compute f(s') = g(s') + h(s')
        if s' ∉ Explored then
            Insert s' into Q with priority f(s')
        end if
    end for
end while
return No solution found
```

Figure 3.8: Algorithm for A* Search

> **Do we add all neighbours, as in BFS?** Generate all legal successors, but apply the duplicate/cost checks before adding them. BFS and DFS skip previously discovered states. UCS and A* keep a cheaper route if one is found. All four select from the *entire frontier*, rather than only from the current node's children.

The A* pseudocode permits a state to be processed again if a better path is discovered: this is **reopening**. Permanently rejecting every state that was expanded earlier can lose the optimal path when an admissible heuristic is inconsistent. With a consistent heuristic, a state does not need reopening after its best entry is expanded.

5. A Trace with a Tie*

Solved Problem 7: Trace A* on a fully specified graph The edges and costs are $S \to A$ (2), $S \to B$ (5), $A \to C$ (4), $B \to C$ (1), and $C \to G$ (3). Use

$$h(S) = 7,\ h(A) = 6,\ h(B) = 3,\ h(C) = 2,\ h(G) = 0.$$

Break equal priorities by insertion order; keep the existing parent when a new route has the same cost.

Step	Selected	g	h	Frontier after processing, shown as $n : f$
1	S	0	7	$[A : 8,\ B : 8]$
2	A	2	6	$[B : 8,\ C : 8]$; $g(C) = 6$ via A.
3	B	5	3	$[C : 8]$; the route via B also costs 6.
4	C	6	2	$[G : 9]$
5	G	9	0	Goal selected; stop.

The stored parent chain gives $\boxed{S \to A \to C \to G}$, cost 9. The route $S \to B \to C \to G$ also costs 9, but equal cost does not change the stored parent under our convention. These heuristic values are consistent on every stated edge.

Solved Problem 8: Computing f-values does not determine a path Suppose the supplied values are

Node	g	h	$f = g + h$
A	0	10	10
B	3	6	9
C	5	4	9
D	7	2	9
G	9	0	9

The arithmetic is straightforward, but the edges and current frontier are missing. We cannot infer a route $A \to B \to C \to D \to G$ from this table. A* compares only nodes that have actually been generated. Also, if the true cheapest remaining cost from A is 9, then $h(A) = 10$ is not admissible.

Optional measures of heuristic quality. The error is $e(n) = h^*(n) - h(n)$. If N counts generated nodes including the root for a depth-d solution, the effective branching factor b^* is defined by $N = 1 + b^* + \cdots + (b^*)^d$. A smaller b^* indicates less search work for comparable problems; calculating it is secondary to tracing A* correctly.

3.3.3 Comparison of Search Strategies

Method	Frontier	Selects by	Complete?	Optimal?
BFS	FIFO queue	Least depth	Yes, finite b and finite-depth goal.	Yes, equal positive step costs.
DFS	LIFO stack	Deepest waiting node	Yes on a finite graph with duplicate detection; not generally.	No.
UCS	Priority queue	Least g	Yes on finite graphs; positive cost bound needed in infinite spaces.	Yes, nonnegative costs.
GBFS	Priority queue	Least h	Yes on finite graphs with duplicate detection; not generally.	No.
A*	Priority queue	Least $g + h$	Yes under the stated search-space and heuristic assumptions.	Yes, admissible h with reopening, or consistent h with closed states.

Memory and time. BFS can require $O(b^d)$ time and space; tree DFS uses $O(b^m)$ time and $O(bm)$ stack space. UCS is governed by cost rather than depth. GBFS and A* can still take exponential time and retain a large frontier; an admissible heuristic does not guarantee polynomial time.

> **Special cases and dead ends.** If $h(n) = 0$ everywhere, A* becomes UCS. With equal positive step costs, UCS follows BFS's depth ordering. A finite cheap branch that cannot reach a goal is eventually exhausted; the search then selects another waiting branch. No rule requires every state to connect to the goal. Infinite branches need the completeness assumptions stated above.

3.4 Optimization

3.4.1 Local Search Techniques

Route search cares about the path. In an optimization task, we may care mainly about the final arrangement: a good timetable, a short tour, or a high-scoring parameter setting.

Let $F(s)$ be an objective score, with larger values preferred:

$$s^* = \arg\max_{s \in S} F(s).$$

Single-state local methods keep a current candidate and examine neighbours, rather than storing a full search tree. Genetic algorithms keep a population. Here F is an objective score; it is separate from the search priority $f = g + h$.

1. Hill Climbing

Steepest-ascent hill climbing moves to the neighbour with the highest score and stops when no neighbour is better.

Algorithm 3.6: Steepest-Ascent Hill Climbing

1: Set *current* $\leftarrow s_0$.
2: **repeat**
3: Generate the neighbours of *current*.
4: If there are no neighbours, return *current*.
5: Let *next* be a neighbour with the largest F.
6: If $F(next) \leq F(current)$, return *current*.
7: Set *current* $\leftarrow$ *next*.

Solved Problem 9: Trace hill climbing and recognize a local optimum The states form a line $A - B - C - D - E$ with scores $3, 5, 8, 6, 4$. Only immediate neighbours are reachable in one move.

Starting at A: move to B because $5 > 3$, then to C because $8 > 5$. At C, neighbours B and D have scores 5 and 6, so stop at C with score 8. This is the best state in the line.

Now change only $F(E)$ to 10. The same run still stops at C with score 8: reaching the better state E requires first accepting the worse state D with score 6. Here C is a *local*, rather than global, maximum.

Three common difficulties:

- **Local optimum:** Better than all available neighbours, but not best overall.
- **Plateau:** Several neighbours have the same score; strictly improving moves may stop.
- **Ridge:** A narrow rising region whose direction does not match the allowed individual moves.

A concrete ridge. Let $F(x, y) = x + y - 2|x - y|$, and allow one-unit horizontal or vertical moves. The diagonal $x = y$ rises: $F(0, 0) = 0$ and $F(1, 1) = 2$. However, each of the four immediate neighbours of $(0, 0)$ has a lower score. Reaching $(1, 1)$ needs two coordinated moves, with a temporary decrease. Strict hill climbing therefore stops despite the rising diagonal ridge.

Variants. Stochastic hill climbing chooses among improving moves; first-choice hill climbing accepts the first improvement found; random-restart hill climbing tries several initial states. These ideas reduce some difficulties but do not give a general finite-run guarantee of reaching the global optimum.

2. Simulated Annealing

Simulated annealing sometimes accepts a worse move so it can escape a local optimum. The **temperature** $T > 0$ controls how often this happens. For maximization, let $\Delta = F(s') - F(s)$:

$$\Pr(\text{accept } s') = \begin{cases} 1, & \Delta \geq 0, \\ \exp(\Delta/T), & \Delta < 0. \end{cases}$$

This is the acceptance probability after a neighbour has been proposed; it is not the complete transition probability of the search process.

Solved Problem 10: The effect of cooling A proposed move lowers the score from 8 to 6, so $\Delta = -2$.

$$T = 2: \ \Pr(\textit{accept}) = e^{-1} \approx 0.368; \qquad T = 0.5: \ \Pr(\textit{accept}) = e^{-4} \approx 0.018.$$

At a higher temperature, the temporary decrease is much easier to accept. As the temperature decreases, the method behaves more like hill climbing. A practical cooling schedule need not find the global optimum.

3. Genetic Algorithms (GAs)

A genetic algorithm maintains a population $P_t = \{s_1^t, \ldots, s_N^t\}$ of candidate solutions, called chromosomes. A fitness score measures how good each candidate is.

Algorithm 3.7: Basic Genetic Algorithm

1: Initialize a population and evaluate each candidate's fitness.
2: **while** the stopping condition is not met **do**
3: **Select** parents, favouring higher fitness.
4: Apply **crossover** to combine parts of the parents.
5: Apply occasional **mutation** to introduce variation.
6: Evaluate offspring and form the next population by **replacement**.
7: Return the best candidate retained.

Solved Problem 11: Roulette-wheel selection probabilities The four fitness scores are $10, 20, 40, 30$, whose sum is 100. Fitness-proportionate selection gives

$$p_i = \frac{F(s_i)}{\sum_j F(s_j)} : \qquad (p_1, p_2, p_3, p_4) = (0.1, 0.2, 0.4, 0.3).$$

Thus the third candidate has the largest selection chance, 40%. The probabilities sum to one. This formula needs nonnegative fitness values and a positive total; raw negative scores must first be transformed appropriately.

Selection choices. Tournament selection compares a small sampled group; rank-based selection uses the candidates' ranking.

Crossover example. Splitting parents `0101|1100` and `1100|0011` after the fourth bit gives offspring `0101|0011` and `1100|1100`. Other choices include two-point and uniform crossover.

Mutation examples. Flip a binary bit, perturb a real-valued parameter, or swap two positions in an ordering. Mutation introduces variation; it does not necessarily improve fitness on each application. Encodings and operators must preserve or repair problem constraints.

3.5 Adversarial Search

In a route problem, the next state depends on our action. In a competitive game, an opponent also chooses actions and tries to make our result worse. We therefore plan for both our moves and the opponent's replies.

3.5.1 Game Theory in AI

For the standard minimax setting, assume a deterministic, turn-taking, two-player, zero-sum game with visible states. **MAX** tries to increase the score; **MIN** tries to decrease the same score. All utilities are measured from MAX's perspective.

$$U_{MAX}(s) + U_{MIN}(s) = 0.$$

For a completed tic-tac-toe game, one possible utility is

$$U(s) = \begin{cases} +1, & \text{MAX wins,} \\ 0, & \text{draw,} \\ -1, & \text{MAX loses.} \end{cases}$$

A game-tree node records a board state and whose turn it is. Its children are the states after legal moves. The numbers at the leaves are outcomes or evaluations; they are *not edge costs*.

3.5.2 Minimax Search Algorithm

Minimax asks: *Which move gives MAX the best result when MIN replies optimally?* Compute values from the leaves upward, alternating minimum and maximum.

Minimax value:

$$V(s) = \begin{cases} U(s), & s \text{ is terminal,} \\ \max_{a \in A(s)} V(T(s,a)), & s \text{ is a MAX node,} \\ \min_{a \in A(s)} V(T(s,a)), & s \text{ is a MIN node.} \end{cases}$$

At a MAX root, choose a move attaining the largest child value.

Algorithm 3.9: Depth-Limited Minimax

Function $Minimax(s, depth, maxTurn)$:
  If s is terminal, return $U(s)$.
  If $depth = 0$, return the evaluation $E(s)$.
  If $maxTurn$ is true, set $v \leftarrow -\infty$; otherwise set $v \leftarrow +\infty$.
  for each legal successor s' **do**
    $w \leftarrow Minimax(s', depth - 1, \neg maxTurn)$.
    If $maxTurn$, set $v \leftarrow \max(v, w)$.
    Otherwise set $v \leftarrow \min(v, w)$.
  Return v.

If terminal states are not reached within the depth limit, an **evaluation function** estimates the state's quality, for example

$$E(s) = w_1\phi_1(s) + \cdots + w_k\phi_k(s).$$

In chess, features might measure material and king safety. A depth-limited evaluation is an estimate, so it does not guarantee the true full-game outcome.

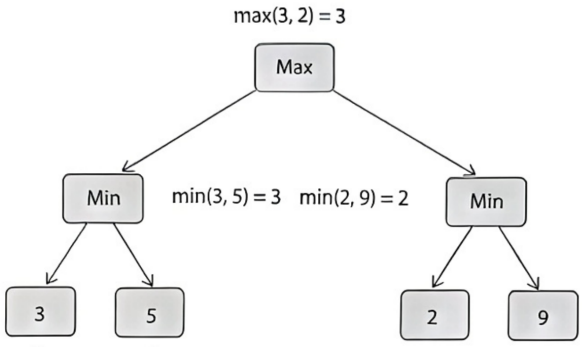


Figure 3.9: The supplied minimax figure: each MIN node takes its smaller leaf value; MAX then chooses the larger returned value.

Solved Problem 12: A complete minimax calculation For the two diagrams above:

1. Left MIN returns $\min(3, 5) = 3$.
2. Right MIN returns $\min(2, 9) = 2$.
3. Root MAX returns $\max(3, 2) = 3$ and chooses the left branch.

The leaf 9 is attractive to MAX, but MAX cannot force MIN to choose it. MIN would select 2 on the right. Thus the *guaranteed* result is better on the left.

3.5.3 Alpha–Beta Pruning

Alpha–beta pruning computes the minimax result while skipping branches that cannot improve the decision at an ancestor.

- $\boldsymbol{\alpha}$ is a lower bound from alternatives already available to MAX along the current path.
- $\boldsymbol{\beta}$ is an upper bound from alternatives already available to MIN along the current path.

Start at the root with $(\alpha, \beta) = (-\infty, +\infty)$. Pass the current bounds to child calls.

At MAX: $\alpha \leftarrow \max(\alpha, v)$.

At MIN: $\beta \leftarrow \min(\beta, v)$.

Cutoff: If $\boxed{\alpha \geq \beta}$, skip the current node's remaining children. The available alternatives at an ancestor already make them irrelevant.

Algorithm 3.10: Alpha–Beta Search

Function $AB(s, depth, \alpha, \beta, maxTurn)$:
  If terminal, return $U(s)$; if $depth = 0$, return $E(s)$.
  if $maxTurn$ **then**
    Set $v \leftarrow -\infty$.
    for each legal successor s' **do**
      $v \leftarrow \max(v, AB(s', depth - 1, \alpha, \beta, false))$.
      Set $\alpha \leftarrow \max(\alpha, v)$.
      If $\alpha \geq \beta$, break the successor loop.
  else
    Set $v \leftarrow +\infty$.
    for each legal successor s' **do**
      $v \leftarrow \min(v, AB(s', depth - 1, \alpha, \beta, true))$.
      Set $\beta \leftarrow \min(\beta, v)$.
      If $\alpha \geq \beta$, break the successor loop.
  Return v.
Start a MAX-root call with $AB(s_0, d, -\infty, +\infty, true)$.

Solved Problem 13: Alpha–beta on the two-branch minimax tree Visit leaves from left to right in Figure **??**.

1. At the left MIN node, inspect 3 and 5; it returns 3. Its incoming $\alpha = -\infty$, so neither leaf causes a cutoff.

2. The root now has a guaranteed alternative of 3: set $\alpha = 3$.

3. Enter the right MIN node with $\alpha = 3$. Inspect its first leaf, 2; its β becomes 2.

4. Since $\beta = 2 \leq \alpha = 3$, skip leaf 9. MIN can already force this branch to be at most 2, which cannot beat MAX's available 3.

The root value remains 3 and MAX chooses left. Minimax evaluates four leaves; alpha–beta evaluates three. After a cutoff, a returned internal value may be a bound rather than the subtree's exact minimax value.

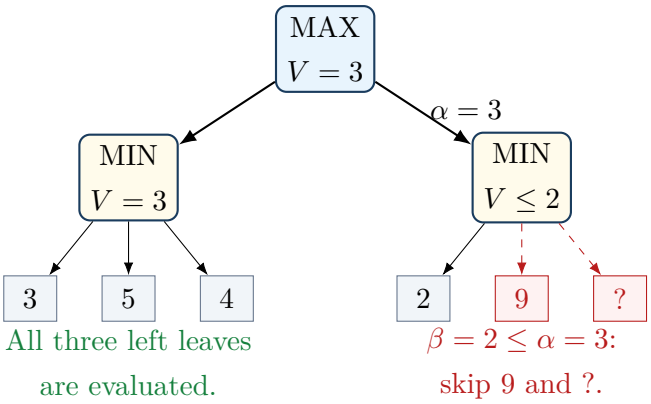


Figure 3.10: Alpha-Beta Pruning.

Solved Problem 14: Count minimax and best-case alpha–beta work A full game tree has branching factor $b = 3$ and depth $d = 3$.

- Full minimax visits $1 + 3 + 9 + 27 = 40$ nodes and evaluates 27 terminal leaves.
- Under perfect move ordering, alpha–beta can visit $1 + 3 + 5 + 11 = 20$ nodes and evaluate 11 leaves. For a full uniform tree, the best-case leaf count is $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1 = 9 + 3 - 1 = 11$.

These are *visited-node* and *evaluated-leaf* counts, respectively. Pruning does not simply halve each level. The actual saving depends on the utilities and the order of moves.

Exam point: With the same depth limit and evaluation function, alpha–beta preserves the minimax value and an optimal move; tied optimal moves can depend on ordering. Worst-case work remains $O(b^d)$. Good ordering can reduce the usual best-case search work to roughly $O(b^{d/2})$, allowing deeper search in the same time.

3.6 Example Problems in Search and Optimization

3.6.1 One Graph, Four Search Strategies

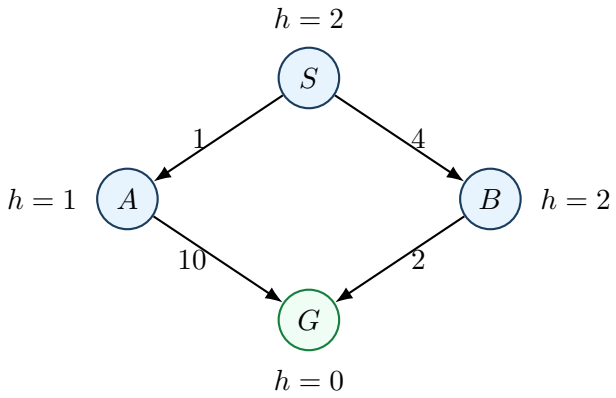


Figure 3.11: Edge labels are actual costs; h-labels estimate remaining costs. The two routes have the same depth but different costs.

Solved Problem 15: Compare BFS, UCS, GBFS and A* step by step Use Figure 3.11. The routes are

$$S \to A \to G: \ 1 + 10 = 11, \qquad S \to B \to G: \ 4 + 2 = 6.$$

Generate A before B. For priority queues, the number after a colon is the priority: g for UCS, h for GBFS and $g + h$ for A*.

Method	After expanding S	After processing A	Next steps and result
BFS	$[A, B]$	Generate G; return immediately.	$S \to A \to G$, cost 11; minimum depth, not minimum cost.
UCS	$[A:1, B:4]$	$[B:4, G:11]$	Select B, improve G to 6, then select G; cost 6.
GBFS	$[A:1, B:2]$	$[G:0, B:2]$	Select G before B; return cost 11.
A*	$[A:2, B:6]$	$[B:6, G:11]$	Select B, improve G to $f = 6$, then select G; cost 6.

The stated heuristic is consistent: for example $h(S) = 2 \leq 1 + h(A)$ and $h(B) = 2 \leq 2 + h(G)$. Even a consistent heuristic does not make *GBFS* optimal because GBFS ignores g.

Should G enter the queue twice? For UCS or A*, the first route generates a G entry with cost 11. The later route improves it to 6. Either update the existing entry, or add the new entry and mark/skip the older one as obsolete. Conceptually, store the *best known route to the state*; do not treat the two costs as interchangeable. The pseudocode above uses the second implementation.

3.6.2 Job Scheduling with Constraints

Let $x_{ij} = 1$ if job j is assigned to machine i, and 0 otherwise. With m machines and n jobs, assigning every job exactly once requires

$$\sum_{i=1}^{m} x_{ij} = 1 \quad (j = 1, \ldots, n), \qquad x_{ij} \in \{0, 1\}.$$

This is only the assignment part of a model. A complete scheduling problem also needs its machine-capacity or timing constraints and, if optimizing, an objective such as minimum completion time. The example illustrates how real tasks become variables and constraints.

3.6.3 Travelling Salesperson Problem (TSP)

Visit each city exactly once and return to the start, minimizing

$$\sum_{i=1}^{n-1} d(c_i, c_{i+1}) + d(c_n, c_1).$$

The nearest-neighbour heuristic repeatedly chooses the closest *unvisited* city, then returns to the start.

Solved Problem 16: Nearest-neighbour tour through five cities The distances are

	A	B	C	D	E
A	0	10	15	20	25
B	10	0	35	40	30
C	15	35	0	50	45
D	20	40	50	0	55
E	25	30	45	55	0

Starting at A, the decisions are:

1. From A, choose B: distance 10.
2. From B, among unvisited C, D, E, choose E: distance 30.
3. From E, among unvisited C, D, choose C: distance 45.
4. Only D remains: move $C \to D$ (50), then return $D \to A$ (20).

The tour is $A \to B \to E \to C \to D \to A$, with

$$\boxed{10 + 30 + 45 + 50 + 20 = 155}.$$

It is not optimal: the valid tour $A \to C \to E \to B \to D \to A$ costs $15 + 45 + 30 + 40 + 20 = 150$. A locally cheapest next move need not yield a globally cheapest tour.

3.6.4 Small Practice Problems with Answers

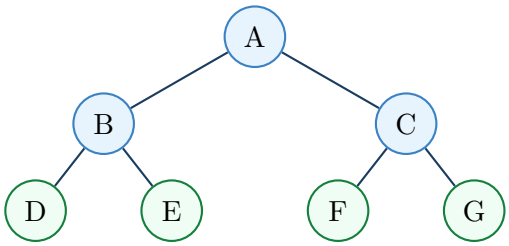


Figure 3.12: The editable BFS/DFS traversal tree retained from the supporting notes, with sibling spacing adjusted for readability.

Solved Problem 17: Write traversal orders Traverse the complete tree in Figure 3.12, left to right.

BFS: A, B, C, D, E, F, G.

DFS: A, B, D, E, C, F, G.

These are visiting orders, not a single root-to-leaf solution path.

Solved Problem 18: Which frontier node does A* select next? The waiting nodes are A with $g = 3, h = 4$ and B with $g = 1, h = 8$.

$$f(A) = 3 + 4 = 7, \qquad f(B) = 1 + 8 = 9.$$

A* selects A. UCS would select B because $1 < 3$; GBFS would select A because $4 < 8$. Compare the priorities required by the stated method.

Solved Problem 19: Back up a MIN node A MIN node has children with utilities $7, 2, 5$. It returns $\min(7, 2, 5) = 2$ to its parent. If the incoming alpha bound is 4 and the first evaluated leaf is 2, the remaining children can be pruned because $2 \leq 4$.

3.7 Revision and Self-Study

3.7.1 Common Mistakes

- **"BFS always finds the cheapest path."** It minimizes the number of moves; this minimizes cost when every move has the same positive cost.
- **"Choose the edge with the smallest cost in UCS."** UCS compares the full accumulated g, not just the newest edge.
- **"Choose the smallest h in A*."** A* uses $g + h$; GBFS uses h.
- **"Stop when UCS or A* first generates a goal."** Wait until the best current goal entry is removed from the priority queue.
- **"Never revisit a discovered state."** UCS and A* must retain a better route; A* may need reopening with an inconsistent heuristic.
- **"Every selected node belongs to the answer path."** Reconstruct the path through parents; other selected nodes were explored alternatives.

- **"A* needs only an admissible heuristic, regardless of the implementation."** Graph search also needs correct cost updates and reopening, or consistency when expanded states are closed permanently.
- **"Count the blank in the 8-puzzle heuristic."** Standard misplaced-tile and Manhattan heuristics count numbered tiles only.
- **"Take MAX at every minimax level."** Alternate MAX and MIN, with all scores from the same player's perspective.
- **"Pruning changes the minimax value."** It avoids work that cannot change the root decision; an internal cutoff may return only a bound.

3.7.2 Short Revision Questions

1. Formulate the 8-puzzle or water-jug problem using states, actions, transitions, a goal test and cost.
2. Distinguish a state-space graph, a search tree, a frontier and an explored set.
3. Explain why BFS uses a queue and DFS uses a stack.
4. When does a shallowest path also have minimum cost?
5. Explain why UCS must update a more expensive route to an already discovered state.
6. Compare $g(n)$, $h(n)$ and $f(n)$ in UCS, GBFS and A*.
7. State admissibility and consistency. Explain why reopening may be needed.
8. Calculate the misplaced-tile and Manhattan heuristics for a small board.
9. Explain a local optimum, a plateau and a ridge; describe how simulated annealing differs from hill climbing.
10. Define a CSP and explain backtracking, MRV and constraint propagation.
11. Evaluate a small minimax tree and identify alpha–beta cutoffs for a stated leaf order.
12. Distinguish a guaranteed minimax value from an expected value at a chance node.

3.8 Quick Revision Sheet

Concept	Remember
Search problem	(S, A, T, s_0, S_g, C); the state space may be implicit in a five-component description.
State versus node	A state is a configuration; a search node also records its path information.
Frontier	Generated nodes waiting to be selected. Expand means generate successors.
BFS	FIFO; least depth first. Complete for finite branching and a finite-depth goal; cost-optimal for equal positive step costs.
DFS	LIFO or recursion; deep branch first. Complete on finite graphs with duplicate detection; not generally optimal.
UCS	Priority $g(n)$. Update cheaper paths and test the goal on removal. Optimal for nonnegative costs.
GBFS	Priority $h(n)$. Ignores the cost already paid; not generally optimal.
A*	Priority $g(n) + h(n)$. Admissible heuristic plus reopening, or consistent heuristic with closed states, supports optimality.
Completeness caveat	Infinite-space cost search usually needs finite branching and costs $\geq \epsilon > 0$; infinite zero-cost paths can obstruct progress.
Admissible	$0 \leq h(n) \leq h^*(n)$ and $h(G) = 0$: never overestimate.
Consistent	$h(n) \leq c(n, n') + h(n')$ on every edge; f does not decrease along a path.
8-puzzle heuristics	Misplaced numbered tiles; sum of numbered-tile Manhattan distances. Exclude the blank.
Hill climbing	Accept improving neighbours; can stop at a local optimum, plateau or ridge.
Simulated annealing	May accept a worse move; lower temperature makes this less likely.
Genetic algorithm	Population → selection → crossover → mutation → replacement.
CSP	Variables, domains, constraints; a solution is a complete consistent assignment.
MRV / degree	Fewest legal values first; ties favour constraints on unassigned variables.
Minimax	MAX takes maximum; MIN takes minimum. Evaluate leaves from MAX's perspective.
Alpha–beta	Start $(-\infty, +\infty)$; MAX updates α, MIN updates β; cut off when $\alpha \geq \beta$.
Chance node	$V = \sum_i p_i V_i$, with probabilities summing to one.

Which node is selected next?

BFS: depth DFS: stack top UCS: g GBFS: h A*: $g + h$.

Which path is returned? Follow the selected goal's parent links back to the start.